

The Conditional Location of a Median Path

Biing-Feng Wang⁺, Shan-Chyun Ku^{*}, and Yong-Hsian Hsieh⁺

⁺ Department of Computer Science, National Tsing Hua University, Taiwan, ROC

^{*} Faraday Technology Corporation, Taiwan, ROC

bfwang@cs.nthu.edu.tw, scku@faraday.com.tw, Eric@venus.cs.nthu.edu.tw

Abstract

In this paper, we study the problem of locating a median path of limited length on a tree under the condition that some existing facilities are already located. The existing facilities may be located at any subset of vertices. Upper and lower bounds are proposed for both the discrete and continuous models. In the discrete model, a median path is not allowed to contain partial edges. In the continuous model, a median path may contain partial edges. The proposed upper bounds for these two models are $O(n \log n)$ and $O(n \log n \alpha(n))$, respectively. They improve the previous ones from $O(n \log^2 n)$ and $O(n^2)$, respectively. The proposed lower bounds are both $\Omega(n \log n)$. The lower bounds show that our upper bound for the discrete model is optimal and the margin for possible improvement on our upper bound for the continuous model is slim.

Key words: location theory, medians, centers, conditional location problems, trees, paths, algorithms

1 Introduction

Network location theory has been traditionally concerned with the optimal location of points. Let $G=(V, E)$ be a graph. For any pair $v, u \in V$, let $d(v, u)$ be their distance. For each node $v \in V$, let $w(v)$ be its vertex weight. The well-known p -median problem is to find a subset H of p vertices minimizing $\sum_{v \in V} \{w(v) \times \min_{u \in H} d(v, u)\}$, which is called the *distance-sum* of H . The well-known p -center problem is to find a subset H of p vertices minimizing $\max_{v \in V} \{w(v) \times \min_{u \in H} d(v, u)\}$, which is called the *eccentricity* of H . Slater [7] firstly extended the network location theory to include a

facility that is not merely a single-point but a path. He defined a *core* of a tree as a path of any length having the minimum distance-sum. Minieka [5] studied the problem of finding a median path of a specified length in a tree. In the problem, a median path should be exactly equal to a given number l . Later, Hakimi *et al.* [2] studied the problem of locating a median path of limited length in a tree. In the problem, a median path should have length not larger than a given number l . Two models were discussed. If a median path is not allowed to contain partial edges, it is referred as the *discrete model*. Otherwise, it is referred as the *continuous model*. Recently, Alstrup *et al.* [1] gave an $O(n \log n)$ time algorithm for the discrete model and an $O(n \log n \alpha(n))$ time algorithm for the continuous model, where α is the inverse of Ackermann's function. With a little modification, their algorithm for the continuous model can be used to find a median path of a specified length as well. In [5], Minieka extended the location theory to include tree-shaped facilities. Results on locating tree-shaped facilities can be found in [5, 8, 9, 11]. Path-shaped and tree-shaped facilities are also called *extensive facilities*. Parallel algorithms for locating different kinds of extensive facilities have also been examined in the literature [10, 11].

Consider the situation that we are going to locate new facilities on a network in which some existing facilities are already located so that customers can be served by the closest facility

whether existing or new. For example, consider a graph $G=(V, E)$ in which there are existing facilities located at the vertices of a subset $S \subseteq V$. The *conditional p -median problem* is to find a subset H of p vertices minimizing the distance-sum of $H \cup S$ and the *conditional p -center problem* is to find a subset H of p vertices minimizing the eccentricity of $H \cup S$. Minieka [4] coined the term *conditional location problem*. Since then many papers dealing with conditional location problems have appeared. Very recently, Tamir *et al.* [9] launched the study on finding the conditional location of an extensive facility of limited length on trees. They discussed both path-shaped and tree-shaped facilities, both eccentricity and distance-sum criteria, and both continuous and discrete models. Therefore, in total eight problems were examined. For most of the problems, sub-quadratic algorithms were presented.

In this paper, we study the problem of finding the conditional location a median path of limited length on a tree. Upper and lower bounds are proposed for both the discrete and continuous models. For the discrete model, the proposed upper bound is $O(n \log n)$. Tamir *et al.* [9] had proposed an $O(n \log^2 n)$ time algorithm for this model and conjectured that the time complexity of their algorithm can be improved to $O(n \log n)$ by using a more efficient implementation. Our upper bound is obtained by such an implementation. For the continuous model, the proposed upper bound is $O(n \log n \alpha(n))$, improving the previous one in [9] from $O(n^2)$. For the discrete model, the proposed lower bound is $\Omega(n \log n)$. It also holds for the unconditional case. Thus, for the discrete model, the upper bound proposed in this paper and the upper bound proposed in [1] for the unconditional case are both optimal. For the continuous model,

the proposed lower bound is also $\Omega(n \log n)$. It shows that the margin for possible improvement on our upper bound for the continuous model is slim.

The remainder of this paper is organized as follows. In the next section, notation and preliminaries are presented. In Section 3, Alstrup *et al.*'s algorithms for the unconditional case are described. Then, in Sections 4 and 5, our algorithms for the conditional case are presented. Finally, in Section 6, lower bounds are proposed.

2. Notation and preliminaries

Let $T = (V, E)$ be free tree. Let $n = |V|$. Each vertex $v \in V$ has a nonnegative weight $w(v)$ and each edge $e \in E$ has a nonnegative length $d(e)$. In this paper, we assume that T is drawn in the Euclidean plane so that each $e \in E$ is a line segment of length $d(e)$ and T is regarded as a closed and connected subset of points in the Euclidean plane. For any two points a, b of T , let $P(a, b)$ be the unique path from a to b , which is a collection of edges and at most two partial edges, and let $d(a, b)$ be the length of $P(a, b)$. A path $P(a, b)$ is *discrete* if both $a, b \in V$, and is *almost discrete* if at least one of a and b is a vertex. If two vertices $i, j \in V$ are neighbors, then by removing the edge (i, j) two subtrees are induced. We denote by T_j^i the subtree containing j . For convenience, for any $i \in V$, we call each $T_j^i, (i, j) \in E$, a *subtree* of i . For any subtree X of T , the vertex set and edge set of X are denoted, respectively, by $V(X)$ and $E(X)$. For easy description of algorithms, sometimes we will orient T into a rooted tree. In such a case, we denote by $p(i)$ the parent of a vertex $i \in V$.

For any vertex $i \in V$ and any subset Y of points of T , the *distance* from i to Y is the shortest distance from i to any point of Y . If $Y = \emptyset$, $d(i, Y) = \infty$. In this

paper, we assume that there is a subset $S \subseteq V$ representing some existing facilities. For a subtree X and any subset Y of points of T , the *conditional weighted distance-sum* from X to Y is $D(X, Y) = \sum_{i \in V(X)} w(i) \times \min\{d(i, Y), d(i, S)\}$. If $X=T$, we simply write $D(Y)$ in place of $D(T, Y)$. Let $\lambda_i = D(i)$ for each $i \in V$ and let $\lambda_e = D(e)$ for each $e \in E$. For each $(i, j) \in E$, define a function $\delta_{(i,j)}$ as $\delta_{(i,j)}(x) = D(P(i, q))$, where $0 \leq x \leq d(i, j)$ and q is the point on (i, j) with $d(i, q) = x$. Note that δ is defined on ordered pairs of neighboring vertices and thus $\delta_{(i,j)} \neq \delta_{(j,i)}$. By definition, $\delta_{(i,j)}(0) = \lambda_i$ and $\delta_{(i,j)}(d(i, j)) = \lambda_{(j,i)}$.

The *median path problem* is to determine a path H of length $\leq l$ in T minimizing $D(H)$, where $l \geq 0$ is a real number. The determined path is called a *median path*. The problem is *unconditional* if $S = \emptyset$, and is *conditional* otherwise. When a median path is allowed to contain partial edges, we refer to the model as the *continuous model*, and otherwise we refer to it as the *discrete model*.

For any piece-wise linear function f , let $\|f\|$ be the number of linear segments of f . For a set F of functions, the *lower envelope* of F is the function Φ defined as $\Phi(x) = \min\{\infty, \min_{f \in F \text{ defined on } x} \{f(x)\}\}$ for all number $x \geq 0$. Hart and Sharir gave the following result.

Theorem 1 [3]: The lower envelope of n linear functions, each defining a line or a line segment, is a piece-wise linear function having $O(n\alpha(n))$ linear segments.

A *decomposition* of T by using a vertex $i \in V$ as a *decomposer* is a pair of two subtrees $T_1 = (V_1, E_1)$ and $T_2 = (V_2, E_2)$ such that $V_1 \cup V_2 = V$, $V_1 \cap V_2 = \{i\}$, and $E_1 \cup E_2 = E$. For any subtree X of T , we call a vertex in X having a neighbor in T outside X a

boundary vertex. A *cluster* of T is a subtree having at most two boundary vertices. For any cluster C , $\beta(C)$ denotes the set of boundary vertices and $\pi(C)$ denotes the path between the boundary vertices. If C has only one boundary vertex, $\pi(C)$ denotes the vertex. Two clusters A and B can be *merged* if they intersect in a single vertex and $A \cup B$ is still a cluster. A *top tree* τ of T is a binary tree with the following properties [1].

1. Each node of τ represents a cluster of T .
2. The leaves of τ represent the edges of T .
3. Each internal node of τ represents the cluster merged from the two clusters represented by its children.
4. The root of τ represents T .
5. The height of τ is $O(\log n)$.

A top tree of T describes a way to recursively decompose T into subtrees having at most two boundary vertices, until each of the subtrees contains only a single edge.

3. Alstrup *et al.*'s algorithms for the unconditional median path problem

3.1 The continuous model

For any subtree C of T , let $Cost(C)$ be the distance-sum of the best path of length $\leq l$ contained in C . That is, $Cost(C) = \min\{D(H) \mid H \text{ is a path of length } \leq l \text{ in } C\}$. Alstrup *et al.*'s algorithm firstly constructs a top tree τ of T in $O(n)$ time. Then, in a bottom-up fashion, it computes $Cost(C)$ for all clusters C represented by the nodes of τ . The computation for the root gets the distance-sum of a median path.

Besides $Cost(C)$, during the computation, for each cluster C , the value $D(\pi(C))$ and a function $\Phi_{b,C}$ for every $b \in \beta(C)$ are computed, where $\Phi_{b,C}(x) = \min\{D(P(b, q)) \mid P(b, q) \text{ is contained in } C, d(b, q) \leq x\}$ (i.e., $\Phi_{b,C}(x)$ is the distance-sum of the best path of length $\leq x$ that extends from b into C). Define for each $(i, j) \in E$ a function $f_{(i,j)}$ as

$$(1) \quad f_{(i,j)}(x) = \begin{cases} \infty & \text{if } x < 0, \\ \delta_{(i,j)}(x) & \text{if } 0 \leq x < d(i, j), \text{ and} \\ \delta_{(i,j)}(d(i, j)) & \text{if } x \geq d(i, j), \end{cases}$$

which is the distance-sum of the best path of length $\leq x$ that extends from i and is contained in the edge (i, j) . Then, we can express $\Phi_{b,C}(x) =$

$$\min_{(i,j) \in E(C) \text{ and } i \text{ is on } P(b,j)} \{D(P(b, i)) + f_{(i,j)}(x - d(b, i)) - \lambda_i\}.$$

Clearly, since $S = \emptyset$, we have $\delta_{(i,j)}(x) = \lambda_i - x \times |V(T_j^i)|$

for $0 \leq x \leq d(i, j)$. Thus, the function $\Phi_{b,C}$ is the lower envelope of $|E(C)|$ piece-wise linear functions. Since $\sum_{(i,j) \in E(C)} \text{ and } i \text{ is on } P(b,j) \|f_{(i,j)}\| = O(|E(C)|)$, we obtain the following.

Lemma 1 [1]: In the unconditional case, for any cluster C and $b \in \beta(C)$, $\Phi_{b,C}$ is a piece-wise linear function having $O(k\alpha(k))$ linear segments, where $k = |E(C)|$.

Alstrup *et al.*'s algorithm is in Figure 1. Since $S = \emptyset$, by using dynamic programming, it is easy to compute λ_i , $\lambda_{(i,j)}$, and $|V(T_j^i)|$ for all $i \in V$ and $(i, j) \in E$ in $O(n)$ time. Using the computed value, each $\delta_{(i,j)}$ and $f_{(i,j)}$ can be computed in $O(1)$ time. The computation for a leaf cluster takes $O(1)$ time. Since $\Phi_{c,A}$ and $\Phi_{c,B}$ are piece-wise linear, Step 10 determines $\min_{0 \leq x \leq l} \{\Phi_{c,A}(x) + \Phi_{c,B}(l-x) - \lambda_c\}$ in $O(\|\Phi_{c,A}\| + \|\Phi_{c,B}\|)$ time. In Step 14, we obtain $\Phi_{b,C}$

from $\Phi_{b,A}$ and $\Phi_{b,B}$ in $O(\|\Phi_{b,A}\| + \|\Phi_{b,B}\|)$ time. From Lemma 1, we conclude that the computation for a merged cluster C in Steps 8-14 requires $O(k\alpha(k))$ time, where $k = |E(C)|$. In each layer of τ , no two clusters C contain the same edge. Thus, the computation for a layer of τ takes $O(n\alpha(n))$ time. Since there are $O(\log n)$ layers, Algorithm 1 performs in $O(n \log n \alpha(n))$ time.

3.2 The discrete model

Algorithm 1 can be applied to the discrete model by using a different definition of $f_{(i,j)}$. Since only vertices can be the ends of a median path, we re-define $f_{(i,j)}$ as follows:

$$(2) \quad f_{(i,j)}(x) = \begin{cases} \infty & \text{if } x < 0, \\ \lambda_i & \text{if } 0 \leq x < d(i, j), \text{ and} \\ \lambda_{(i,j)} & \text{if } x \geq d(i, j). \end{cases}$$

With the new definition, each $f_{(i,j)}$ is a step function and thus $\|\Phi_{b,C}\| = O(|E(C)|)$ for any cluster C and $b \in \beta(C)$. Therefore, the computation in Algorithm 1 for a cluster C takes $O(|E(C)|)$ time. We conclude that the problem of finding an unconditional discrete median path can be solved in $O(n \log n)$ time.

4. The conditional discrete median path problem

It is easy to see that using the definition of $f_{(i,j)}$ in (2), Algorithm 1 solves the conditional discrete median path problem as well. By a careful check, it can be found that except Step 1, all steps in the algorithm are irrelevant to S . Thus, in the conditional case, those steps also takes $O(n \log n)$ time. In the remainder of this section, we show that Step 1 can be implemented in the same time.

Algorithm 1. Median_Path(T, l)

Input: a tree $T=(V, E)$ and a length $l \geq 0$

Output: the distance-sum of a median path having length $\leq l$

begin

```

1  preprocess  $T$  to obtain  $\lambda_i, \lambda_{(i,j)}$ , and  $f_{(i,j)}$  for all  $i \in V$  and  $(i, j) \in E$ 
2   $\tau \leftarrow$  a top tree of  $T$ 
3  for each leaf cluster  $e=(i, j) \in E$  do
4       $Cost(e) \leftarrow \min\{f_{(i,j)}(l), f_{(j,i)}(l)\}$ 
5       $D(\pi(e)) \leftarrow \begin{cases} \lambda_i & \text{if } \beta(e) = \{i\} \\ \lambda_j & \text{if } \beta(e) = \{j\} \\ \lambda_e & \text{if } \beta(e) = \{i, j\} \end{cases}$ 
6      for each  $b \in \beta(e)$  do  $\Phi_{b,e} \leftarrow \begin{cases} f_{(i,j)} & \text{if } b = i \\ f_{(j,i)} & \text{if } b = j \end{cases}$ 
7  for each cluster  $C$  represented by an internal node of  $\tau$  do (in a bottom-up fashion)
8       $(A, B) \leftarrow$  the two clusters represented by the children of the internal node
9       $c \leftarrow$  the intersection vertex of  $A$  and  $B$ 
10      $H_c \leftarrow \min_{0 \leq x \leq l} \{\Phi_{c,A}(x) + \Phi_{c,B}(l-x) - \lambda_c\}$  // The distance-sum of the best path passing  $c$ 
11      $Cost(C) \leftarrow \min\{Cost(A), Cost(B), H_c\}$ 
12      $D(\pi(C)) \leftarrow \begin{cases} \lambda_b & \text{if } |\beta(C)| = 1 \text{ and } \beta(C) = \{b\} \\ D(\pi(A)) + D(\pi(B)) - \lambda_c & \text{otherwise} \end{cases}$ 
13     for each  $b \in \beta(C)$  do
14          $\Phi_{b,C}(x) \leftarrow \begin{cases} \min\{\Phi_{b,A}(x), \Phi_{b,B}(x)\} & \text{if } b = c \\ \min\{\Phi_{b,A}(x), \Phi_{b,B}(x - d(b, c)) + D(\pi(A)) - \lambda_c\} & \text{if } b \in \beta(A) \setminus \{c\} \\ \min\{\Phi_{b,A}(x - d(b, c)) + D(\pi(B)) - \lambda_c, \Phi_{b,B}(x)\} & \text{if } b \in \beta(B) \setminus \{c\} \end{cases}$ 
15 return( $Cost(T)$ )
end

```

Figure 1. Alstrup *et al.*'s algorithm.

Step 1 is to compute $\lambda_i, \lambda_{(i,j)}$, and $f_{(i,j)}$ for all $i \in V$ and $(i, j) \in E$. For each $(i, j) \in E$, let $m_{i,j} = D(T_j^i, i)$, which by definition is

$$\sum_{v \in V(T_j^i)} w(v) \times \min\{d(v, S), d(v, i)\}.$$

Using $m_{i,j}$, we can compute $\lambda_i = \sum_{(i,j) \in E} \{m_{i,j}\}$ for all $i \in V$ and then compute $\lambda_{(i,j)} = \lambda_i + \lambda_j - m_{i,j} - m_{j,i}$ for all $(i, j) \in E$ in $O(n)$ time. Using λ_i and $\lambda_{(i,j)}$, the functions $f_{(i,j)}$ for all $(i, j) \in E$ can be computed in $O(n)$ time according to the definition in (2). Thus, our problem becomes the computation of all $m_{i,j}$, ($i, j) \in E$

We do the computation of $m_{i,j}$ by using the divide-and-conquer strategy. For convenience, we also describe it on a top tree τ of T . For each cluster C represented by a node of τ , we will compute $m_{i,j}(C) = D(C_j^i, i)$ for each $(i, j) \in E(C)$, and we will compute two arrays $X_{b,C}$ and $Y_{b,C}$ for each $b \in \beta(C)$, where $X_{b,C}$ stores the ordering of the vertices $v \in V(C)$ by the distances $d(v, b)$ and $Y_{b,C}$ stores the ordering of the vertices $v \in V(C)$ by $d(v, S) - d(v, b)$. After the computation, we have $m_{i,j}(T) = m_{i,j}$ for every $(i, j) \in E$. The computation is in Figure 2.

Procedure Computing_m(T)

begin

```

1 preprocess  $T$  to obtain  $d(v, S)$  for all  $v \in V$ 
2  $\tau \leftarrow$  a top tree of  $T$ 
3 for each leaf cluster  $e=(i, j) \in E$  do
4    $m_{i,j}(e) \leftarrow w(j) \times \min\{d(j, S), d(j, i)\}$ 
5    $m_{j,i}(e) \leftarrow w(i) \times \min\{d(i, S), d(i, j)\}$ 
6   for each  $b \in \beta(e)$  do
7      $X_{b,e} \leftarrow \begin{cases} (i, j) & \text{if } b = i \\ (j, i) & \text{if } b = j \end{cases}$ 
8      $Y_{b,e} \leftarrow \begin{cases} (i, j) & \text{if } d(i, S) - d(i, b) \leq d(j, S) - d(j, b) \\ (j, i) & \text{otherwise} \end{cases}$ 
9 for each cluster  $C$  represented by an internal node of  $\tau$  do (in a bottom-up fashion)
10    $(A, B) \leftarrow$  the two clusters represented by the children of the internal node
11    $c \leftarrow$  the intersection vertex of  $A$  and  $B$ 
12   for each  $(i, j) \in E(C)$  do compute  $m_{i,j}(C)$  and  $m_{j,i}(C)$ 
13   for each  $b \in \beta(C)$  do
14     compute  $X_{b,C}$  and  $Y_{b,C}$ 

```

end

Figure 2. Computing $m_{i,j}$.

The computation of $d(v, S)$ for all $v \in V$ in Step 1 takes $O(n \log n)$ time [9]. The computation for a leaf cluster takes $O(1)$ time. Thus, Steps 3-8 takes $O(n)$ time. Consider a fixed merged cluster C . Step 12 computes $m_{i,j}(C)$ and $m_{j,i}(C)$ for all $(i, j) \in E(C)$. Due to the symmetry between A and B , we only present the computation for $(i, j) \in E(A)$. For easy discussion, we assume that C is rooted at c . For each $(i, p(i)) \in E(A)$, we have $C_i^{p(i)} = A_i^{p(i)}$ and $C_{p(i)}^i = A_{p(i)}^i \cup B$. Thus, for all $(i, j) = (i, p(i)) \in E(A)$, we simply compute $m_{j,i}(C) = m_{j,i}(A)$ in $O(|V(C)|)$ time. Next, we describe the computation of $m_{i,j}(C)$ for all $(i, j) = (i, p(i)) \in E(A)$. Assume that $X_{c,A} = (x_1, x_2, \dots, x_s)$ and $Y_{c,B} = (y_1, y_2, \dots, y_t)$. For any vertex $x \in V(A)$, the ordering of the vertices $y \in V(B)$ by $d(y,$

$S) - d(y, x)$ is the same with the ordering by $d(y, S) - d(y, c)$. Thus, for any vertex x_k in $X_{c,A}$, there exists an index z_k such that $d(y_{z_k}, S) - d(y_{z_k}, x_k) \leq 0$ for $q < z_k$ and $d(y_q, S) - d(y_q, x_k) > 0$ for $q \geq z_k$. By applying a process similar to merge, we compute all z_k in $O(|V(C)|)$ time from $X_{c,A}$ and $Y_{c,B}$. Then, since the indices z_k are nondecreasing, it is easy to compute $m_{i,j}(C)$ for all $(i, j) = (i, p(i)) \in E(A)$ in $O(|V(C)|)$ time by using the following equation:

$$\begin{aligned}
 m_{x_k, p(x_k)}(C) &= m_{x_k, p(x_k)}(A) \\
 &\quad + \sum_{q < z_k} w(y_q) \times d(y_q, S) \\
 &\quad + \sum_{q \geq z_k} w(y_q) \times d(y_q, x_k).
 \end{aligned}$$

Therefore, Step 12 takes $O(|V(C)|)$ time. Now, consider the computation in Step 14. By symmetry, we may assume $b \in V(A)$. Clearly, we can obtain

$X_{b,C}$ by merging $X_{b,A}$ and $X_{c,B}$ in linear time. Similarly, $Y_{b,C}$ can be obtained by merging $Y_{b,A}$ and $Y_{c,B}$ in linear time. Therefore, the computation in Steps 10-14 takes $O(|V(C)|)$ time in total, from which we conclude that the whole computation on τ takes $O(n \log n)$ time. We have the following theorem.

Theorem 2: The problem of finding a conditional discrete median path of length $\leq l$ in a tree can be solved in $O(n \log n)$ time.

5. The conditional continuous median path problem

By using the following important property, Tamir *et al.* presented an $O(n^2)$ time algorithm without applying the divide-and-conquer strategy.

Lemma 2 [9]: There is a conditional continuous median path that is almost discrete.

From now on, only almost discrete paths are considered. For easy description, in the remainder of this section, we assume that T is rooted at an arbitrary vertex $r \in V$. Let $P(v, q)$ be an almost discrete path, where v is a vertex and q is a point. The point q is possibly a vertex. For convenience, we say that $P(v, q)$ is of *type 1* if q is on the path from v to r , and is of *type 2* otherwise. For easy description, a discrete path between two vertices $v_1, v_2 \in V$ is regarded as two different almost discrete paths, one extending from v_1 to v_2 and the other extending from v_2 to v_1 , such that any almost discrete path is either of type 1 or of type 2. Our strategy for determining a median path is as follows. Firstly, we compute a best path among all

paths of type 1. Then, we compute a best path among all paths of type 2. Finally, we determine a median path by simply comparing the two computed paths.

5.1 The computation of a best path among all paths of type 1

Let $R = \{P(v, r) \mid v \in V, d(v, r) \leq l\}$ and $Q = \{P(v, q) \mid v \in V, d(v, r) > l, q \text{ is the point on } P(v, r) \text{ with } d(v, q) = l\}$. Our problem in this section is to compute the best path in $R \cup Q$. Since $|R \cup Q| = O(n)$, it is not difficult to derive an $O(n \log n)$ time algorithm for the computation. For easy analyzing, instead of giving a new algorithm, we do the computation by performing an algorithm for the conditional discrete median path problem as follows. Firstly, we compute all the points q with $P(v, q) \in Q$, which requires $O(n \log n)$ time [9]. Then, repeatedly, for each computed point q , letting (i, j) be the edge containing q , we introduce a new vertex v_q at q . Finally, we perform an algorithm for the conditional discrete median path problem. Note that the above computation may produce a path not in $R \cup Q$, which is of type 2.

5.2 The computation of a best path among all paths of type 2

In Algorithm 1, the function $\delta_{(i, p(i))}$ is used to determine the distance-sum of a path having a partial edge extending from i to $p(i)$. Conversely, the function $\delta_{(p(i), i)}$ is used to determine the distance-sum of a path having a partial edge extending from $p(i)$ to i . Now, we are only interested in paths of type 2. Therefore, we obtain an algorithm by slightly modifying Algorithm 1 as follows. Firstly, for each $(i, p(i)) \in E$, we redefine f_i ,

$p(i))(x) = \infty$ for all x , and $f_{(p(i), i)}(x) =$

$$\begin{cases} \infty & \text{if } x < 0, \\ \delta_{(p(i), i)}(x) & \text{if } 0 \leq x < d(p(i), i), \text{ and} \\ \lambda_{(p(i), i)} & \text{if } x \geq d(p(i), i). \end{cases}$$

Next, we modify Steps 10-11 of Algorithm 1 for the computation of $Cost(C)$. Since only almost discrete paths are considered, we compute $Cost(C) = \min\{Cost(A), Cost(B), H_{A,B}, H_{B,A}\}$, where $H_{A,B} = \min_{v \in V(A), d(c,v) \leq l} \{D(P(c, v)) + \Phi_{c,B}(l - d(c, v)) - \lambda_c\}$ is the distance-sum of the best path of type 2 extending from a vertex in A to a point in B , and $H_{B,A} = \min_{v \in V(B), d(c,v) \leq l} \{D(P(c, v)) + \Phi_{c,A}(l - d(c, v)) - \lambda_c\}$ is the distance-sum of the best path of type 2 extending from a vertex in B to a point in A . It is not difficult to see that after the above modification, the computation for a cluster C can be implemented in $O(|V(C)| + k'\alpha(k'))$ time, where $k' = \sum_{(p(i), i) \in E(C)} \|\delta_{(p(i), i)}\|$.

Lemma 3: The total number of linear segments of all $\delta_{(p(i), i)}$, $(p(i), i) \in E$, is $O(n)$.

Lemma 4: All $\delta_{(p(i), i)}$, $(p(i), i) \in E$, can be computed in $O(n \log n)$ time.

Due to the page limitation, the proofs of Lemmas 3 and 4 are omitted. From the lemmas, it is not difficult to obtain the following theorem.

Theorem 3: The conditional continuous mediana path problem can be solved in $O(n \log n \alpha(n))$ time.

6. Lower bounds

Lemma 5 [6]: Solving the element uniqueness problem, which is to decide if any two of n given positive numbers are equal, requires $\Omega(n \log n)$ time

in the comparison model.

Theorem 4: Finding an unconditional discrete median path of length $\leq l$ in a tree requires $\Omega(n \log n)$ time in the comparison model, even for a tree with all vertex weights being equal to 1.

Proof: Given an instance $A = (a_1, a_2, \dots, a_n)$ of the element uniqueness problem, we construct in linear time a tree $T = (V, E)$ with $V = \{r, x_1, y_1, z_1, x_2, y_2, z_2, \dots, x_n, y_n, z_n\}$ and $E = \{(r, x_1), (x_1, y_1), (x_1, z_1), (r, x_2), (x_2, y_2), (x_2, z_2), \dots, (r, x_n), (x_n, y_n), (x_n, z_n)\}$. All vertex weights are 1. Let $t = 3 \times \max\{a_1, a_2, \dots, a_n\}$. The length of (r, x_i) is t . The lengths of (x_i, y_i) and (x_i, z_i) are a_i and $t - a_i$, respectively. Finally, let $l = 3t$. With some efforts, we can show that there are two equal elements in A iff a discrete median path of T has distance-sum $\leq t(4n - 7)$. Therefore the theorem holds. Q.E.D.

Theorem 5: Finding a conditional continuous median path of length $\leq l$ in a tree requires $\Omega(n \log n)$ time in the comparison model, even for a tree with all vertex weights being equal to 1.

Proof: Given an instance $A = (a_1, a_2, \dots, a_n)$ of the element uniqueness problem, we construct in linear time a tree $T = (V, E)$ with $V = \{r, x_1, y'_1, y_1, z'_1, z_1, x_2, y'_2, y_2, z'_2, z_2, \dots, x_n, y'_n, y_n, z'_n, z_n\}$ and $E = \{(r, x_1), (x_1, y'_1), (y'_1, y_1), (x_1, z'_1), (z'_1, z_1), (r, x_2), (x_2, y'_2), (y'_2, y_2), (x_2, z'_2), (z'_2, z_2), \dots, (r, x_n), (x_n, y'_n), (y'_n, y_n), (x_n, z'_n), (z'_n, z_n)\}$. All vertex weights are 1. Let $t = 3 \times \max\{a_1, a_2, \dots, a_n\}$. The length of (r, x_i) is t . The lengths of (x_i, y'_i) , (y'_i, y_i) , (x_i, z'_i) , and (z'_i, z_i) are $a_i/2$, $a_i/2$, $(t - a_i)/2$, and $(t - a_i)/2$, respectively. The existing facility S includes all x_i, y'_i , and z'_i , $i = 1, 2, \dots, n$. Let $l = 3t$. With some efforts, we can show that there are two equal elements in A iff a

conditional continuous median path of T has
distance-sum $\leq t(n-1)/2$. Q.E.D.

References

- [1] S. Alstrup, P.W. Lauridsen, P. Sommerlund, and M. Throup, "Finding cores of limited length," in Proceedings of the 5th International Workshop on Algorithms and Data Structures, Lecture Notes in Computer Science, vol. 1272, Springer, pp. 45-54, 1997.
- [2] S. L. Hakimi, E. F. Schmeichel and M. Labbé, "On locating path- or tree- shaped facilities on networks," *Networks*, vol. 23, pp. 543-555, 1993.
- [3] S. Hart and M. Sharir, "Nonlinearity of Davenport-Schinzel sequences and of general path compression schemes," *Combinatorica*, vol. 6, pp. 151-177, 1986.
- [4] E. Minieka, "Conditional centers and medians on a graph," *Networks*, vol. 10, pp.265-272, 1980.
- [5] E. Minieka, "The optimal location of a path or tree in a tree network," *Networks*, vol. 15, pp. 309-321, 1985.
- [6] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*, Springer-Verlag, 1985.
- [7] P. J. Slater, "Locating central paths in a network," *Transportation Science*, vol. 16, No. 1, pp. 1-18, 1982.
- [8] A. Tamir, "Fully polynomial approximation schemes for locating a tree-shaped facility: a generalization of the knapsack problem," *Discrete Applied Mathematics*, vol. 87, pp. 229-243, 1998.
- [9] A. Tamir, J. Puerto, J.A. Mesa, and A.M. Rodriguez-Chia, "Conditional location of path and tree shaped facilities on trees," manuscript, 2001.
- [10] B.-F. Wang, "Finding a two-core of a tree in linear time," *SIAM Journal on Discrete Mathematics*, accepted.
- [11] B.-F. Wang, "Efficient parallel algorithms for optimally locating a path and a tree of a specified length in a weighted tree network," *Journal of Algorithms*, vol. 34, pp. 90-108, 2000.